

Basic Information about Scalene:

- Scalene is a profiler works solely on python code and tracks CPU and GPU usage not only line-by-line but at the method level as well. (We can provide a quick demo on the web based UI if necessary)
- Scalene provides a lot of visual data on their web based application and as we've previously seen information in a json or html file.
- Made by UMass Amherst's Plasma Lab Create

Advanced Information about Scalene:

- **General Description:**
 - Scalene's CPU profiling differentiates between the time spent in Python and the time spent in native code. NumPy and TensorFlow, two of the more popular libraries, have bottlenecks occurring outside of pure Python code. Therefore, we can assume this distinction contributes a lot to the accuracy of the profiling.
 - Scalene's memory profiling tracks both Python and native memory usage, identifies leaks and excess memory allocations too. Measures usage in native extensions (for instance C code called from Python (which encompasses a large majority of ML python libraries))
 - Claims to report with 10-20% overhead (most accurate on the web right now) and has almost no slowdown when utilizing just CPU and GPU tracking but a 35% slow down when using other features
- **Cool features:**
 - Works with Pytest, pycharm and Django
 - Handles threads and Multiprocessing

Profiling Information

(Note: Most of this comes directly from Scalene Git and a lot is repeated from the general description above, but some in fresh info)

- **CPU profiling**
 - Scalene separates out time spent in Python from time in native code (including libraries). Most Python programmers aren't going to optimize the performance of native code (which is usually either in the Python implementation or external libraries), so this helps

developers focus their optimization efforts on the code they can actually improve.

- Scalene highlights hotspots (code accounting for significant percentages of CPU time or memory allocation) in red, making them even easier to spot.
- Scalene also separates out system time, making it easy to find I/O bottlenecks.
- **GPU profiling:**
 - Scalene reports GPU time (currently limited to NVIDIA-based systems).
- **Memory profiling:**
 - Scalene profiles memory usage. In addition to tracking CPU usage, Scalene also points to the specific lines of code responsible for memory growth. It accomplishes this via an included specialized memory allocator.
 - Scalene separates out the percentage of memory consumed by Python code vs. native code.
 - Scalene produces per-line memory profiles.
 - Scalene identifies lines with likely memory leaks.
 - Scalene profiles copying volume, making it easy to spot inadvertent copying, especially due to crossing Python/library boundaries (e.g., accidentally converting numpy arrays into Python arrays, and vice versa)

How to use scalene:

- **Directly from python**
 - Pip install -U scalene
 - Then simply call, scalene your_script.py
 - Generates detailed report
- **Alternatively:**

```
from scalene import scalene_profiler

# Turn profiling on
scalene_profiler.start()

# your code

# Turn profiling off
scalene_profiler.stop()

from scalene.scalene_profiler import enable_profiling

with enable_profiling():
    # do something
```

- Can be installed as a VS extension and used on the command line
- Works on Linux with Ubuntu (Instructions on git for command line installation)

Additional Documentation:

- If we plan to use Scalene there is an extremely detailed paper published - [Triangulating Python Performance Issues with Scalene](#). This goes in more in-depth on the methodology which we can pick apart for assurance. From a top level glance, a common theme is that they use sampling (Pausing execution and resuming) and program counter tracking to calculate usage.